

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

**COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA0101
AND EXPERT SYSTEMS**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A Modelling Comparative Analysis Assignment

Code of Conduct:

*I **Mangali Veera Naga Sai (192425253)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Veera Naga Sai

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities

	of each approach.
--	-------------------

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Cybersecurity Threat Detection Using Logic and Prolog

1. Problem Statement & Objectives:

Problem Statement:

A Security Operations Center (SOC) needs an automated system to identify possible cybersecurity threats from user and network activities.

The system observes activities such as:

- Repeated login failures
- Unusual login locations
- Privilege escalation
- Suspicious file access
- Abnormal network traffic

Using these observations, the system should determine whether a security threat exists and identify the possible type of threat.

Objectives:

1. Identify suspicious cybersecurity activities.
2. Represent security knowledge using production rules.
3. Model the same knowledge using propositional logic.
4. Represent entities and relationships using first-order logic.
5. Implement the rules using Prolog.
6. Demonstrate forward and backward chaining.
7. Compare the four modelling approaches.
8. Provide an explainable security conclusion.

Possible Conclusions

Brute Force Attack

Suspicious Login

Privilege Escalation Attack

Unauthorized File Access

Network Attack

Security Threat

Normal Activity

2. Domain Knowledge & Facts

Domain

The domain is **Cybersecurity / Security Operations Center (SOC)**.

Entities

Entity	Examples
User	Alice, Bob
Computer	Server1, PC1
IP Address	192.168.1.10
Location	India, USA
File	payroll.pdf
Network	Internal network
Event	Login, file access, privilege change

Important Attributes

- Number of login failures
- Login location
- User's normal location
- Privilege level
- File sensitivity
- Network traffic volume
- Time of activity

Basic Facts

For example:

User Bob had 10 failed login attempts.

Bob logged in from an unusual location.

Bob accessed a sensitive file.

Bob's privileges were increased.

Network traffic was abnormally high.

These facts are given to the inference system.

3. Production Rule Model

Production rules represent knowledge in the form:

IF condition THEN conclusion

Rule 1 – Brute Force Attack

IF login failures are greater than 5
THEN brute force attack is suspected.

Rule 2 – Suspicious Login

IF login location is unusual
AND login failures are high
THEN suspicious login is detected.

Rule 3 – Privilege Escalation

IF a normal user suddenly gets administrator privileges
THEN privilege escalation is suspected.

Rule 4 – Suspicious File Access

IF a user accesses a sensitive file
AND the user is unauthorized
THEN suspicious file access is detected.

Rule 5 – Network Attack

IF network traffic is abnormally high
THEN network attack is suspected.

Rule 6 – Security Threat

IF brute force attack is suspected
THEN security threat is detected.
IF privilege escalation is suspected
THEN security threat is detected.
IF suspicious file access is detected
THEN security threat is detected.
IF network attack is suspected
THEN security threat is detected.

Example

Given:

login_failures = 10

Apply:

IF login_failures > 5

THEN brute_force_attack

Conclusion:

Brute Force Attack Suspected

4. Propositional Logic Model

In propositional logic, each statement is represented using a proposition that is either **True** or **False**.

Propositions

Let:

F = Repeated login failures occurred

L = Login occurred from unusual location

P = Privilege escalation occurred

A = Suspicious file access occurred

N = Abnormal network traffic occurred

B = Brute force attack

T = Security threat

Logical Rules

Rule 1

$F \rightarrow B$

Meaning:

If repeated login failures occur, then a brute-force attack is suspected.

Rule 2

$L \wedge F \rightarrow \text{Suspicious_Login}$

Rule 3

$P \rightarrow \text{Privilege_Escalation_Threat}$

Rule 4

$A \rightarrow \text{Unauthorized_Access}$

Rule 5

$N \rightarrow \text{Network_Attack}$

Overall Threat

$B \vee \text{Privilege_Escalation_Threat}$

$\vee \text{Unauthorized_Access}$

$\vee \text{Network_Attack}$

$\rightarrow T$

Example

Given:

$F = \text{True}$

and:

$F \rightarrow B$

Therefore:

$B = \text{True}$

Since:

$B \rightarrow T$

we conclude:

$T = \text{True}$

Limitation

Propositional logic treats facts as complete propositions. It cannot naturally represent different users, computers, files, and relationships.

For example:

Bob accessed payroll.pdf

is treated as one proposition rather than representing **Bob**, **payroll.pdf**, and the **access relationship** separately.

5. First-Order Logic Model

First-order logic (FOL) is more expressive because it represents **objects, properties, relationships, and variables**.

Predicates

FailedLogin(User, Count)

UnusualLocation(User)

AdminPrivilege(User)

NormalUser(User)

Accessed(User, File)

Sensitive(File)

Unauthorized(User, File)

HighTraffic(Network)

Rule 1 – Brute Force

$\forall x (\text{FailedLogin}(x, n) \wedge n > 5$

$\rightarrow \text{BruteForce}(x))$

Meaning:

For every user x, if the user has more than five failed login attempts, the user may be involved in a brute-force attack.

Rule 2 – Suspicious Login

$\forall x (\text{UnusualLocation}(x) \wedge \text{BruteForce}(x))$
 $\rightarrow \text{SuspiciousLogin}(x))$

Rule 3 – Privilege Escalation

$\forall x (\text{NormalUser}(x) \wedge \text{AdminPrivilege}(x))$
 $\rightarrow \text{PrivilegeEscalation}(x))$

Rule 4 – Unauthorized File Access

$\forall x \forall f (\text{Unauthorized}(x,f) \wedge \text{Sensitive}(f))$
 $\rightarrow \text{SuspiciousFileAccess}(x,f))$

Rule 5 – Network Attack

$\forall n (\text{HighTraffic}(n))$
 $\rightarrow \text{NetworkAttack}(n))$

Rule 6 – Security Threat

$\forall x (\text{BruteForce}(x) \rightarrow \text{Threat}(x))$
 $\forall x (\text{PrivilegeEscalation}(x) \rightarrow \text{Threat}(x))$
 $\forall x (\text{SuspiciousFileAccess}(x,f) \rightarrow \text{Threat}(x))$

Example

Facts:

$\text{FailedLogin}(\text{Bob}, 10)$

$\text{UnusualLocation}(\text{Bob})$

Rule:

$\text{FailedLogin}(\text{Bob}, 10) \wedge 10 > 5$
 $\rightarrow \text{BruteForce}(\text{Bob})$

Then:

$\text{BruteForce}(\text{Bob})$

$\rightarrow \text{Threat}(\text{Bob})$

Therefore:

Bob is associated with a security threat.

6. Prolog Model & Implementation

Prolog is particularly suitable because it naturally represents **facts, rules, and logical queries**.

Prolog Facts

```
failed_login(bob, 10).  
unusual_location(bob).  
normal_user(bob).  
admin_privilege(bob).  
sensitive(payroll).  
unauthorized(bob, payroll).  
high_traffic(network1).
```

Prolog Rules

Brute Force

```
brute_force(User) :-  
    failed_login(User, Count),  
    Count > 5.
```

Suspicious Login

```
suspicious_login(User) :-  
    unusual_location(User),  
    brute_force(User).
```

Privilege Escalation

```
privilege_escalation(User) :-  
    normal_user(User),  
    admin_privilege(User).
```

Suspicious File Access

```
suspicious_file_access(User, File) :-  
    unauthorized(User, File),  
    sensitive(File).
```

Network Attack

```
network_attack(Network) :-  
    high_traffic(Network).
```

Overall Threat

```
threat(User) :-  
    brute_force(User).
```

```
threat(User) :-  
    suspicious_login(User).
```

```
threat(User) :-  
    privilege_escalation(User).
```

```
threat(User) :-  
    suspicious_file_access(User, _).
```

Queries

Query 1

```
?- brute_force(bob).
```

Output:

```
true.
```

Query 2

```
?- suspicious_login(bob).
```

Output:

```
true.
```

Query 3

```
?- privilege_escalation(bob).
```

Output:

```
true.
```

Query 4

```
?- suspicious_file_access(bob, payroll).
```

Output:

```
true.
```

Query 5

```
?- threat(bob).
```

Output:

```
true.
```

7. Forward & Backward Chaining

A. Forward Chaining

Forward chaining starts with **facts** and moves toward a conclusion.

Given Facts

Failed login = 10

Unusual location = Yes

Reasoning

FailedLogin(Bob,10)



Count > 5



BruteForce(Bob)



Threat(Bob)

Therefore:

Security Threat Detected

Forward Chaining Algorithm

START



Read all security facts



Check rules



If rule conditions are satisfied



Generate new fact



Check rules again



Threat detected?



Display result

END

B. Backward Chaining

Backward chaining starts with a **goal** and works backward to find supporting facts.

Suppose the goal is:

Threat(Bob)

The system asks:

How can Threat(Bob) be true?

It finds:

BruteForce(Bob) $\rightarrow$ Threat(Bob)

Then asks:

How can BruteForce(Bob) be true?

It finds:

FailedLogin(Bob, Count)

Count > 5

Available fact:

FailedLogin(Bob,10)

Since:

10 > 5

the goal is proven.

Backward Reasoning

Goal: Threat(Bob)

↓

Need: BruteForce(Bob)

↓

Need: FailedLogin(Bob, Count)

↓

Fact: FailedLogin(Bob,10)

↓

10 > 5

↓

BruteForce(Bob)

↓

Threat(Bob)

Important Point

Prolog primarily uses backward chaining.

8. Comparative Analysis

Feature	Production Rules	Propositional Logic	First-Order Logic	Prolog
Representation	IF–THEN rules	Propositions	Predicates + variables	Facts + rules
Expressiveness	Medium	Low	High	High
Variables	Limited	No	Yes	Yes
Relationships	Limited	Poor	Excellent	Excellent
Inference	Rule matching	Logical inference	Logical inference	Backward chaining
Backtracking	Usually no	No	Depends on system	Yes
Complexity	Low–Medium	Low	High	Medium–High
Scalability	Medium	Low	High but complex	Good for rule systems
Flexibility	Good	Limited	Very good	Very good
Explainability	High	High	High	High
Suitable for SOC	Good	Basic	Very good	Very good

Simple Understanding

Production Rules

Best when you want:

IF this happens → THEN this threat exists.

Propositional Logic

Best for:

Simple True/False security conditions.

First-Order Logic

Best when you have:

Many users, files, computers, and relationships.

Prolog

Best when you want:

An executable expert system that can automatically reason over facts and rules.

9. Results, Discussion & Recommended Model

Sample Test Results

Test Case Observed Facts		Expected Result	Result
TC1	10 failed logins	Brute Force	Detected
TC2	Unusual location + failed logins	Suspicious Login	Detected
TC3	Normal user + admin privilege	Privilege Escalation	Detected
TC4	Unauthorized access to sensitive file	Suspicious File Access	Detected
TC5	Abnormally high network traffic	Network Attack	Detected

Discussion

The system successfully derives security conclusions from observed activities.

For example:

10 failed logins

↓

Brute Force

↓

Security Threat

Similarly:

Normal User

+

Admin Privilege

↓

Privilege Escalation

↓

Security Threat

Recommended Model

For the proposed cybersecurity expert system, **Prolog with first-order logic concepts** is the most suitable approach.

Why?

- Supports variables.
- Represents different users and files.
- Handles relationships.
- Supports logical queries.
- Provides backward reasoning.

- Supports backtracking.
- Rules can be easily modified.
- Suitable for explainable rule-based security systems.

Limitations

The system has some limitations:

1. It depends on predefined rules.
2. It cannot automatically discover completely new attack patterns.
3. Real cybersecurity data is much more complex.
4. False positives may occur.
5. Real-time network monitoring requires integration with external security tools.

Future Enhancements

The system can be improved by adding:

- Machine learning for anomaly detection
- Real-time SIEM integration
- IP reputation checking
- Threat-intelligence feeds
- User behavior analytics
- Automatic alert generation
- Database support
- Web-based SOC dashboard

10. Conclusion, References & GitHub

Conclusion

The **Cybersecurity Threat Detection System** demonstrates how a real-world security problem can be represented using different AI knowledge-representation techniques.

The same cybersecurity knowledge was modelled using **production rules, propositional logic, first-order logic, and Prolog**. Production rules provide a simple IF–THEN representation, propositional logic handles basic Boolean conditions, first-order logic provides greater expressiveness using variables and relationships, and Prolog converts the knowledge into an executable reasoning system.

Both **forward and backward chaining** were demonstrated. Forward chaining begins with observed security events and derives threats, while backward chaining begins with a suspected threat and searches for facts that prove it. Prolog is recommended as the practical

implementation approach because it supports logical reasoning, unification, variables, and backtracking.

References

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
2. Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
3. SWI-Prolog Documentation.
4. William F. Clocksin and Christopher S. Mellish, *Programming in Prolog*.
5. NIST, *Computer Security Incident Handling Guide*.

GitHub Repository

Create a repository with a structure such as:

Cybersecurity-Threat-Detection/

```
|
|— cybersecurity.pl
|— production_rules.txt
|— propositional_logic.txt
|— first_order_logic.txt
|— test_cases.txt
|— screenshots/
|   |— input.png
|   |— output.png
|— README.md
```

GitHub Link: Add your actual repository link after uploading the project.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				